

Evolutionary algorithms

Principles, current research status, applications, libraries and experiments

Julia Rodríguez Gómez

1. Introduction - basic ideas

Evolutionary algorithms (EAs) are optimization methods inspired by biological evolution, such as natural selection, reproduction, mutation, and survival of the fittest [1]. They are used to solve complex optimization problems where the search space is large, non-linear, or difficult to explore with traditional methods.

In evolutionary algorithms, a population of candidate solutions is maintained. Each individual represents a possible solution, usually encoded as a vector of parameters (a chromosome). Each solution is evaluated using a fitness function, which measures how good it is for the given problem. The best individuals are selected to generate new solutions using genetic operators such as crossover and mutation. This process is repeated over multiple generations to gradually improve the population [2]. A key aspect of this approach is the balance between exploration of new regions of the search space and exploitation of already good solutions.

Genetic Algorithms (GAs), introduced by Holland, are one of the most popular types of evolutionary algorithms. They have been successfully applied in areas such as feature selection, neural network optimization, scheduling problems, and hyperparameter tuning in machine learning.

In supervised learning, model performance depends heavily on hyperparameters such as learning rate, regularization strength, number of estimators, or network architecture. These parameters are not learned during training, so they must be optimized separately. Traditional methods such as grid search and random search are simple but often inefficient, especially when the number of hyperparameters increases. Genetic algorithms provide an alternative because they can explore the search space more globally and are less likely to get stuck in local optima [3].

In this project, we apply a genetic algorithm to optimize hyperparameters of supervised learning models, using validation performance (e.g., accuracy or F1-score) as the fitness function.

2. Evolutionary algorithm operation principle

A basic evolutionary algorithm works by iteratively improving a population of candidate solutions.

First, an initial population is created randomly. Each individual represents a possible solution to the problem, such as a set of hyperparameters. Then, each individual is evaluated using a fitness function, which measures its performance (for example accuracy, F1-score, or validation loss in machine learning tasks).

After evaluation, a selection process chooses the best individuals to act as parents. Common selection methods include tournament selection, roulette wheel selection, and rank-based selection. The goal is to give better solutions a higher probability of being selected, while still preserving diversity in the population to avoid premature convergence.

New solutions are generated using crossover and mutation. Crossover combines genetic information from two parents to create offspring, while mutation introduces small random changes to individual solutions. Mutation is especially important in hyperparameter optimization because it helps explore new configurations that may not be reachable through recombination alone.

Finally, a replacement strategy determines how the new generation replaces the old population, either fully or partially. This process is repeated until a stopping criterion is met, such as reaching a maximum number of generations, a time limit, or convergence of the population.

3. Current research status and application areas – review of selected current research works

Recent research in evolutionary computation has focused heavily on hyperparameter optimization (HPO) and Automated Machine Learning (AutoML), since these problems involve large, complex, and often mixed-type search spaces where traditional optimization methods are less effective.

A relevant research direction is the development of hybrid methods that combine genetic algorithms with techniques such as Bayesian optimization or surrogate models. These approaches aim to reduce the computational cost of evaluating candidate solutions by approximating the fitness function, while still preserving strong exploration capabilities [4]. This is especially important in machine learning, where model training is computationally expensive.

Another important application area is deep learning and neural architecture search (NAS). In these approaches, evolutionary algorithms are used not only to optimize hyperparameters such as learning rate or batch size, but also to design neural network architectures. This research direction, often referred to as neuroevolution, has shown promising results in complex tasks where manual design is difficult or suboptimal [5], [9].

In addition, evolutionary algorithms have been applied in reinforcement learning, where they are used to optimize policy parameters and training configurations of models such as Deep Q-Networks (DQN). These methods can improve training stability and performance, especially in environments with sparse or delayed rewards [7].

Some recent works also explore multi-objective evolutionary algorithms (MOEAs), where multiple objectives (e.g., accuracy vs. model complexity) are optimized simultaneously. A well-known example is NSGA-II, which is widely used in optimization problems requiring trade-offs between conflicting objectives [8].

Overall, current research highlights several key trends:

- Genetic algorithms are effective in continuous, discrete, and mixed search spaces.
- Hybrid optimization methods improve efficiency and scalability.
- Applications extend from classical machine learning to deep learning, neuroevolution, and reinforcement learning.
- Multi-objective evolutionary algorithms are increasingly used to handle trade-offs in real-world problems.

Despite these advantages, computational cost remains the main limitation due to repeated model evaluations, which motivates ongoing research into more efficient and hybrid approaches.

4. Review of selected programming libraries and tools

Several libraries are available for implementing genetic algorithms in machine learning and optimization problems.

DEAP (Distributed Evolutionary Algorithms in Python)

DEAP is a widely used Python library for evolutionary computation. It provides tools to implement genetic algorithms in a flexible way, allowing users to define custom fitness functions, selection methods, crossover, and mutation operators. It integrates well with machine learning libraries such as scikit-learn and TensorFlow, which makes it particularly suitable for hyperparameter optimization tasks. It also supports parallel execution, which is important when fitness evaluations are computationally expensive.

Jenetics

Jenetics is a Java-based evolutionary computation library designed with a functional programming approach. It supports genetic algorithms, genetic programming, and multi-objective optimization. Its main advantages include strong type safety, modular design, and good performance, which make it suitable for large-scale or industrial applications.

Comparison

DEAP is more commonly used in academic and research environments because of its integration with the Python data science ecosystem. Jenetics is often preferred in enterprise applications where Java is the main technology stack. Both libraries are suitable for hyperparameter optimization, but DEAP is more frequently used in machine learning research due to its simplicity and flexibility.

5. Selected optimization problem

In this project, the selected optimization problem is the tuning of hyperparameters of a supervised learning model using a Genetic Algorithm (GA). The objective is to identify a set of hyperparameters that maximizes the model's performance on a validation dataset.

The model considered in this work is a Random Forest classifier, which is widely used due to its robustness, ability to handle heterogeneous data, and strong performance in classification tasks. However, its effectiveness depends heavily on the choice of hyperparameters, making it a suitable candidate for optimization.

The hyperparameters selected for optimization include the number of trees in the forest (`n_estimators`), the maximum depth of each tree (`max_depth`), and the minimum number of samples required to split a node (`min_samples_split`). These parameters directly influence the trade-off between underfitting and overfitting, and different configurations can lead to significantly different model performance.

For the experimental evaluation, a standard benchmark dataset (e.g., the Iris dataset) will be used. The dataset is divided into training and validation subsets, allowing the evaluation of each hyperparameter configuration.

The objective of the optimization process is to maximize classification accuracy on the validation set, which is used as the fitness function in the genetic algorithm. Each individual in the population represents a candidate set of hyperparameters, and its fitness corresponds to the performance of the model trained with those parameters.

To assess the effectiveness of the proposed approach, the results obtained using the Genetic Algorithm will be compared with those achieved using Random Search, a commonly used baseline method for hyperparameter optimization.

This problem is particularly relevant because hyperparameter tuning is a critical step in machine learning workflows. Traditional methods such as grid search or random search often become inefficient as the search space grows. In contrast, genetic algorithms provide a flexible global optimization strategy, making them well-suited for exploring complex and high-dimensional search spaces.

6. The implemented algorithm

In this project, a Genetic Algorithm (GA) is implemented to optimize the hyperparameters of a Random Forest classifier. The algorithm follows an evolutionary approach in which a population of candidate solutions evolves over several generations in order to maximize classification accuracy on a validation dataset.

Solution representation

Each individual in the population represents a candidate solution consisting of a set of hyperparameters of the Random Forest model. The individual is encoded as a vector of three values:

- number of trees (n_estimators)
- maximum depth of the trees (max_depth)
- minimum number of samples required to split a node (min_samples_split)

Each parameter is initialized randomly within predefined ranges. This representation allows the algorithm to explore different configurations of the model.

Fitness function

The fitness of each individual is evaluated by training a Random Forest classifier using the corresponding hyperparameters and measuring its classification accuracy on the validation dataset. The objective of the algorithm is to maximize this accuracy.

The fitness function is computed using 5-fold cross-validation accuracy in order to ensure a robust estimation of model performance.

Selection method

Tournament selection is used to choose individuals for reproduction. In each tournament, a small subset of individuals is randomly selected from the population, and the individual with the highest fitness is chosen as a parent. This method promotes better solutions while maintaining diversity.

Crossover operator

A two-point crossover operator is applied to generate offspring. This operator exchanges sections of the hyperparameter vector between two parent individuals, enabling the combination of good characteristics from different solutions.

Mutation operator

Mutation is applied with a predefined probability to introduce variability into the population. During mutation, one or more hyperparameters of an individual are randomly modified within their allowed ranges. This helps prevent premature convergence and improves exploration of the search space.

Algorithm parameters

The main parameters of the genetic algorithm are defined as follows:

- population size: 10 individuals
- number of generations: 5
- crossover probability: 0.5
- mutation probability: 0.2
- tournament size: 3

These parameters were selected to provide a balance between exploration of the search space and computational efficiency, considering the cost of training the model for each individual.

7. Results of experiments

In this project, the performance of the proposed Genetic Algorithm (GA) was evaluated and compared with a Random Search approach. Additionally, a baseline Random Forest model with fixed hyperparameters was used as a reference.

The experiments were conducted by executing both the Genetic Algorithm and Random Search multiple times (5 runs each). This approach allows us to account for the stochastic nature of the algorithms and obtain reliable performance metrics. The evaluation of each solution was based on classification accuracy using cross-validation.

7.1 Experimental Results

The following results were obtained:

Genetic Algorithm (GA):

Best accuracy: 0.9778
Worst accuracy: 0.9722
Average accuracy: 0.9733

Random Search:

Best accuracy: 0.9833
Worst accuracy: 0.9722
Average accuracy: 0.9767

The baseline model achieved an accuracy of approximately 0.9720.

Figure 1 shows the comparison between the Genetic Algorithm and Random Search across multiple runs. The chart illustrates the performance variability and highlights the consistency of both methods.

7.2 Analysis and Discussion

The results show that both optimization methods achieve high performance, with only small differences between them. In this case, Random Search obtained slightly higher best and average accuracy compared to the Genetic Algorithm.

However, the Genetic Algorithm demonstrated stable and consistent performance across multiple runs, producing results close to the optimal values in all cases. This indicates that the evolutionary approach is effective in exploring the hyperparameter search space.

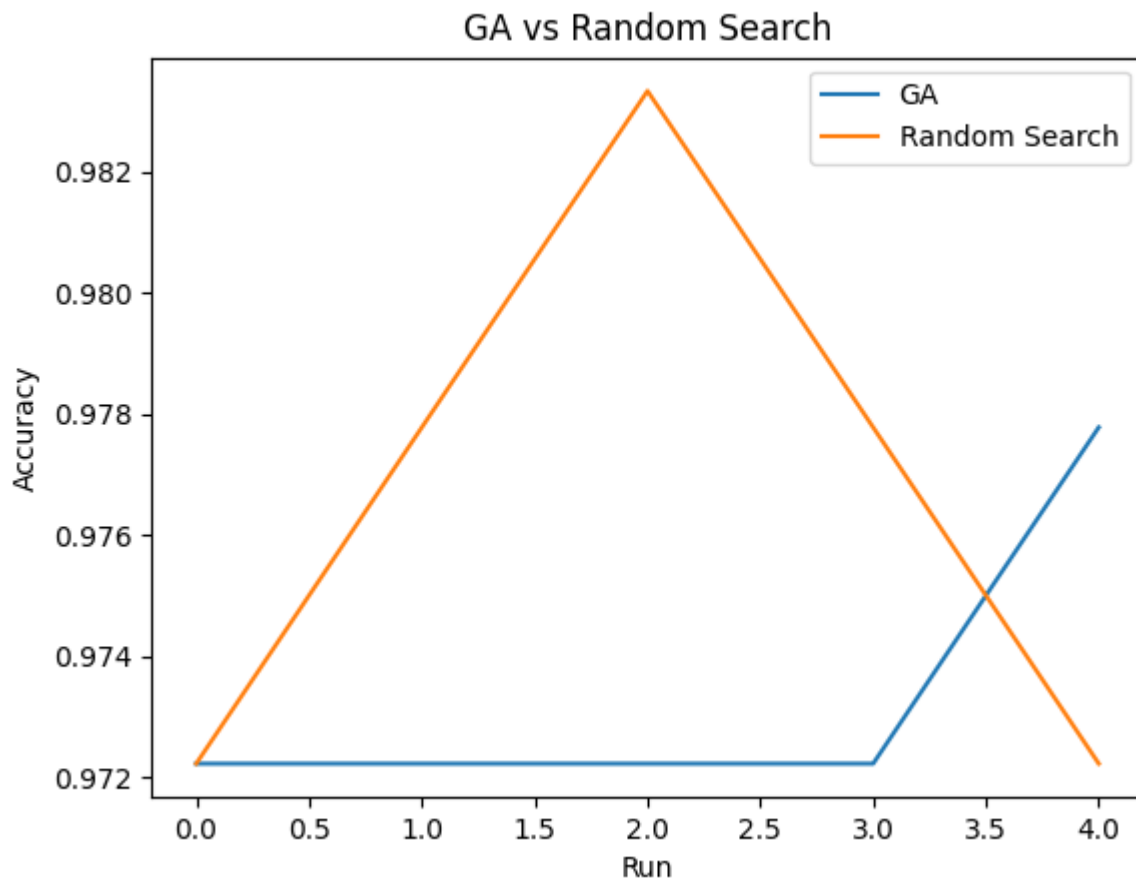
The relatively small performance differences can be explained by the simplicity of the dataset used in this project. Since the dataset is not highly complex, many different hyperparameter configurations can achieve similar levels of accuracy. As a result, the advantage of more advanced optimization methods is less pronounced.

In more complex machine learning problems, especially those with larger datasets or higher-dimensional search spaces, Genetic Algorithms are expected to outperform simpler methods like Random Search due to their ability to balance exploration and exploitation.

7.3 Summary

In conclusion, both Random Search and the Genetic Algorithm provided competitive results for hyperparameter optimization. While Random Search achieved slightly better performance in this specific case, the Genetic Algorithm remains a powerful and flexible optimization technique, particularly suitable for more complex scenarios.

Figure 1: Comparison of GA and Random Search performance across multiple runs.



8. Summary

This project investigated the use of Genetic Algorithms for hyperparameter optimization in supervised learning. A Genetic Algorithm was implemented to optimize the hyperparameters of a Random Forest classifier and its performance was compared with Random Search and a baseline model.

The experiments showed that both optimization methods achieved high accuracy results. In this case, Random Search obtained slightly better performance, although the Genetic Algorithm also demonstrated stable and competitive results across multiple runs.

The project combined theoretical concepts of evolutionary algorithms with a practical implementation using Python and the DEAP library. Overall, the work demonstrated that Genetic Algorithms are a flexible and effective optimization technique for machine learning problems, especially for more complex search spaces.

Bibliography

- [1] Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- [2] Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
- [3] Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*.
- [4] Hutter, F., Kotthoff, L., & Vanschoren, J. (2019). *Automated Machine Learning: Methods, Systems, Challenges*. Springer.
- [5] Stanley, K. O., & Miikkulainen, R. (2002). Evolving neural networks through augmenting topologies (NEAT). *Evolutionary Computation*.
- [6] Real, E. et al. (2017). Large-scale evolution of image classifiers. *ICML*.
- [7] Salimans, T. et al. (2017). Evolution Strategies as a scalable alternative to reinforcement learning. *arXiv:1703.03864*.
- [8] Deb, K. et al. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*.
- [9] Elsken, T., Metzen, J. H., & Hutter, F. (2019). Neural architecture search: A survey. *JMLR*.